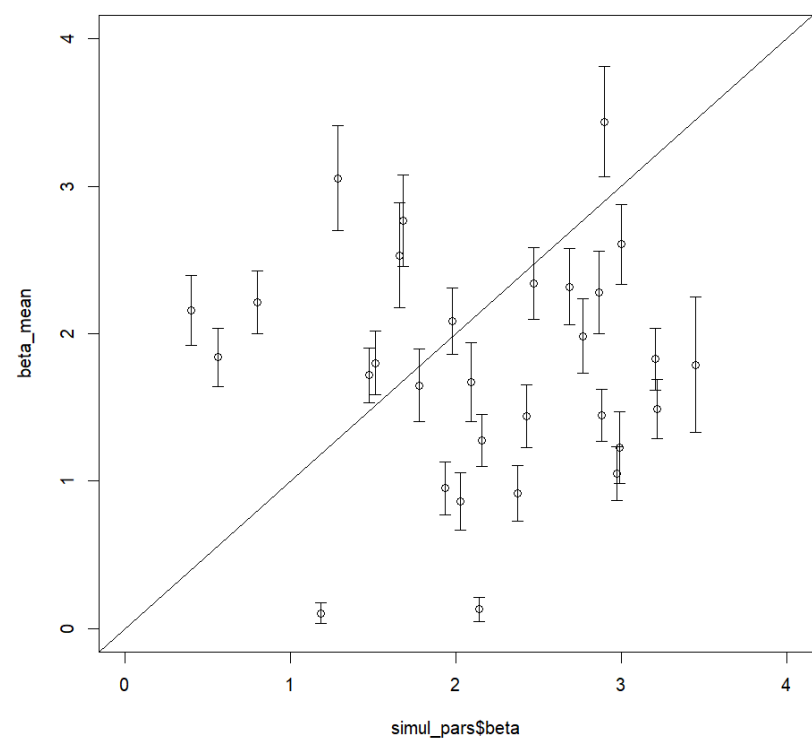
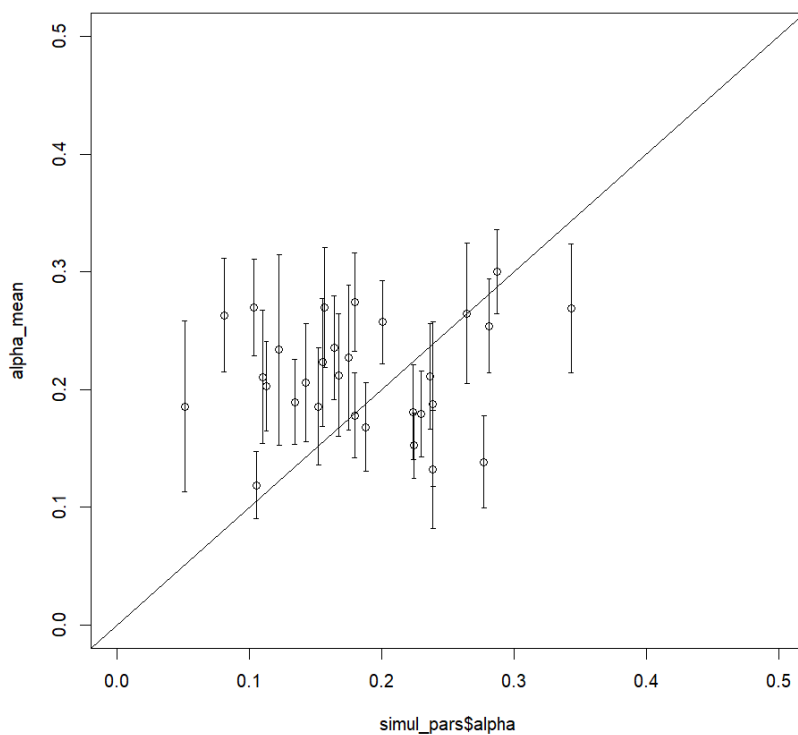


Question 1

- Scatterplot of true and estimated parameters with error bars: Plots indicate alpha was recovered relatively well compared to beta. The scatter plots show dots relatively clustered around the identity line, indicating true parameters were correctly recovered to a certain degree. The parameter values seem to have been pulled towards the group mean (0.20 for alpha, 2.00 for beta).

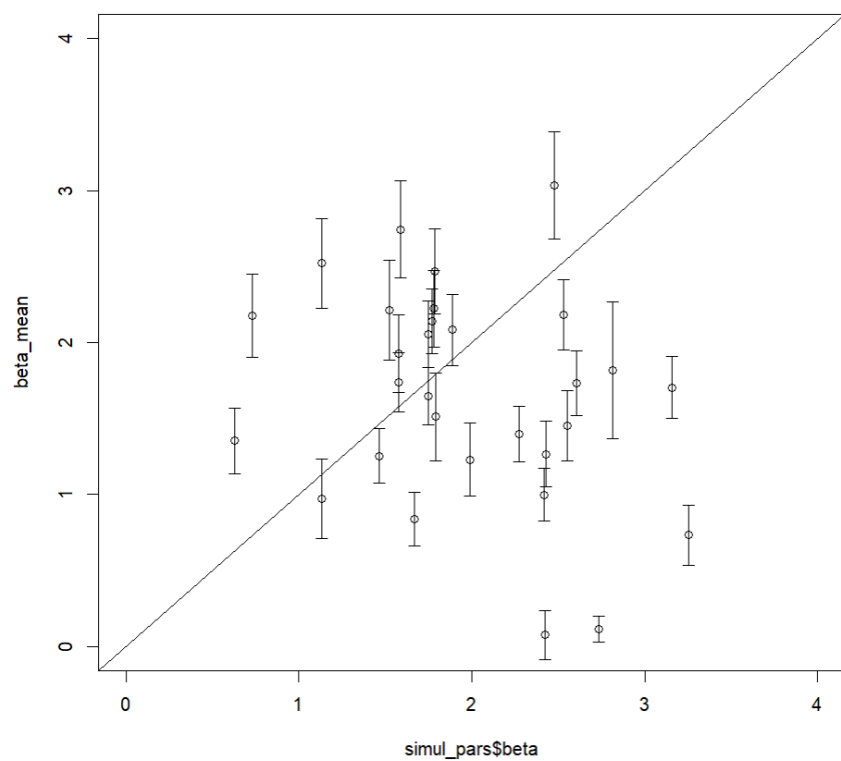
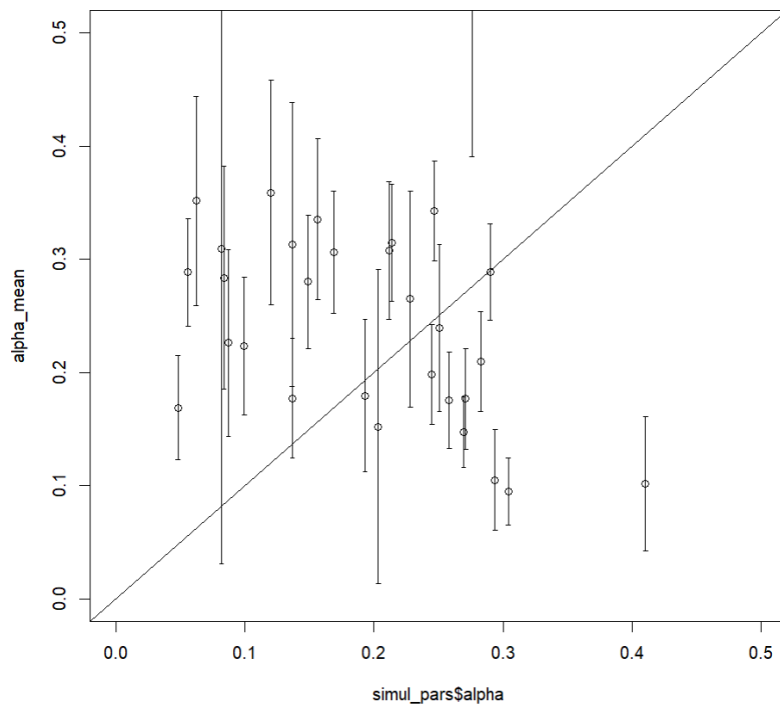


- Additionally, posterior summary for the group parameters

	mean	se_mean	sd	2.5%	50%	97.5%	n_eff	Rhat
mu_alpha	0.21	0	0.02	0.17	0.21	0.24	880	1

	mean	se_mean	sd	2.5%	50%	97.5%	n_eff	Rhat
mu_beta	1.65	0.01	0.21	1.25	1.65	2.08	268	1

Question 2



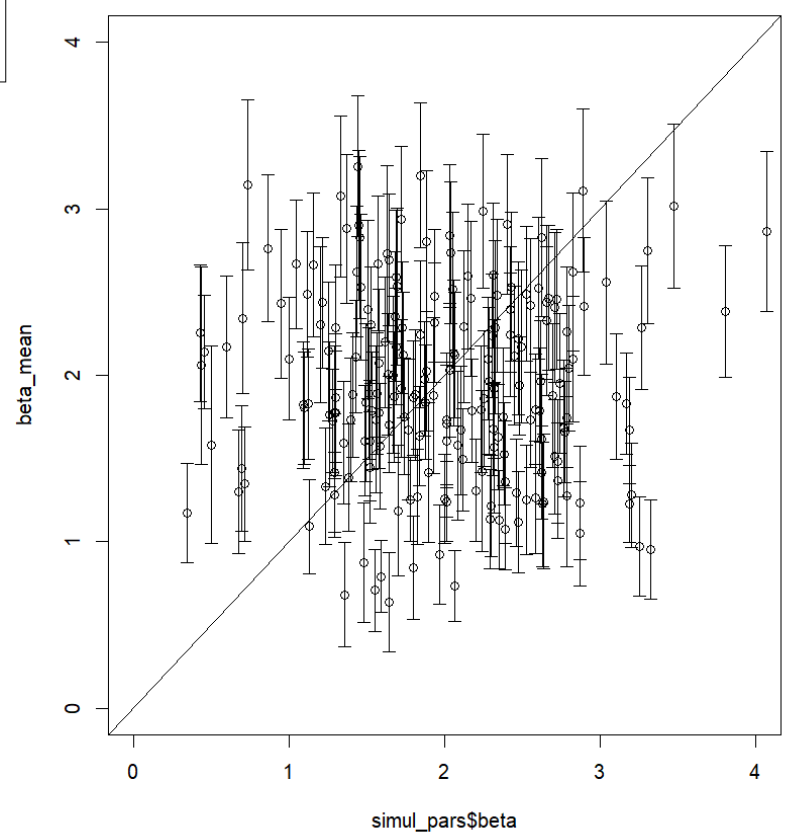
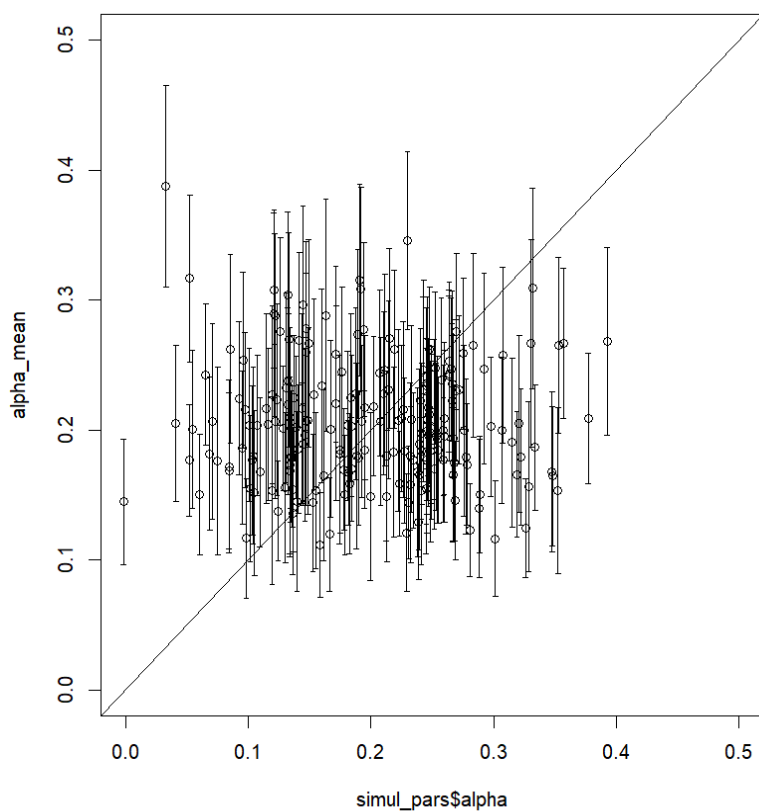
The scatterplots suggest that the non-hierarchical Bayesian model does worse in recovering the true parameters compared to the hierarchical Bayesian model. The estimates are further away from the identity line in the bottom plots (Question 2), and the error bars are noticeably larger, especially for α , indicating greater posterior uncertainty. The estimated parameters

are also more dispersed along the y-axis for the non-hierarchical model, which likely occurred because the parameter estimates were less compressed towards the group mean while in the hierarchical model, shrinkage pulled the parameter estimates closer to the group mean.

Question 3

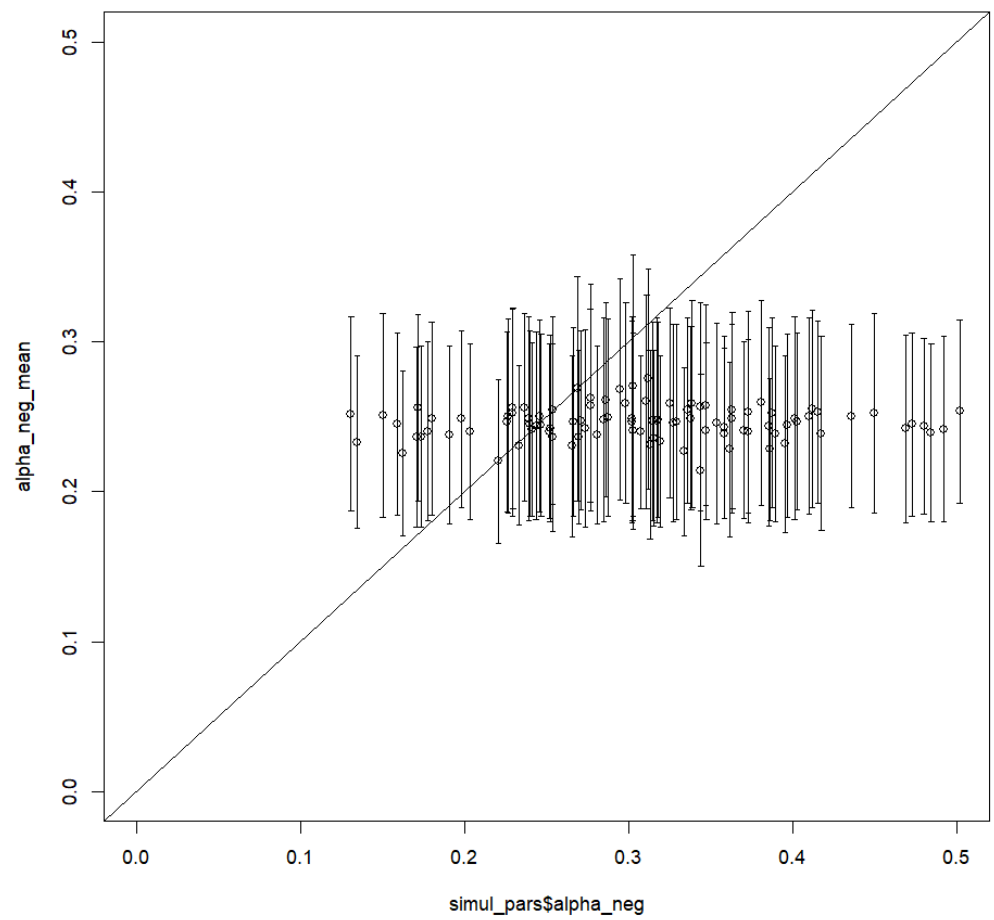
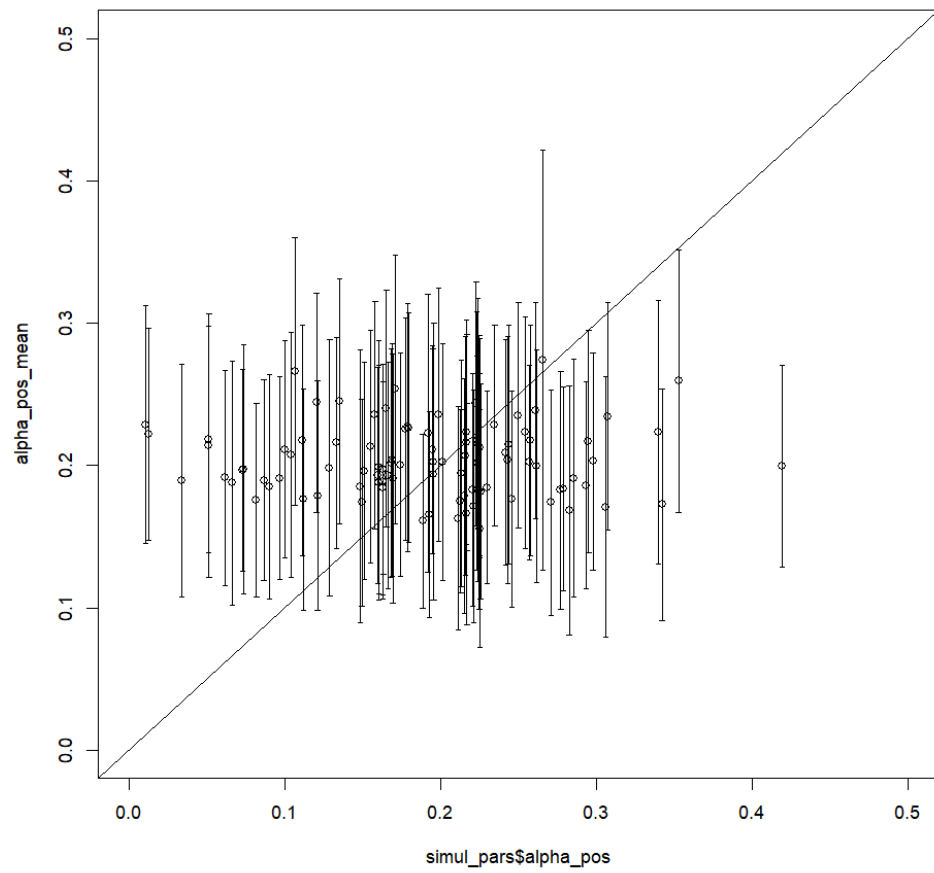
The scatter plots show dots relatively clustered around the identity line, indicating the true parameters were correctly recovered to a certain degree.

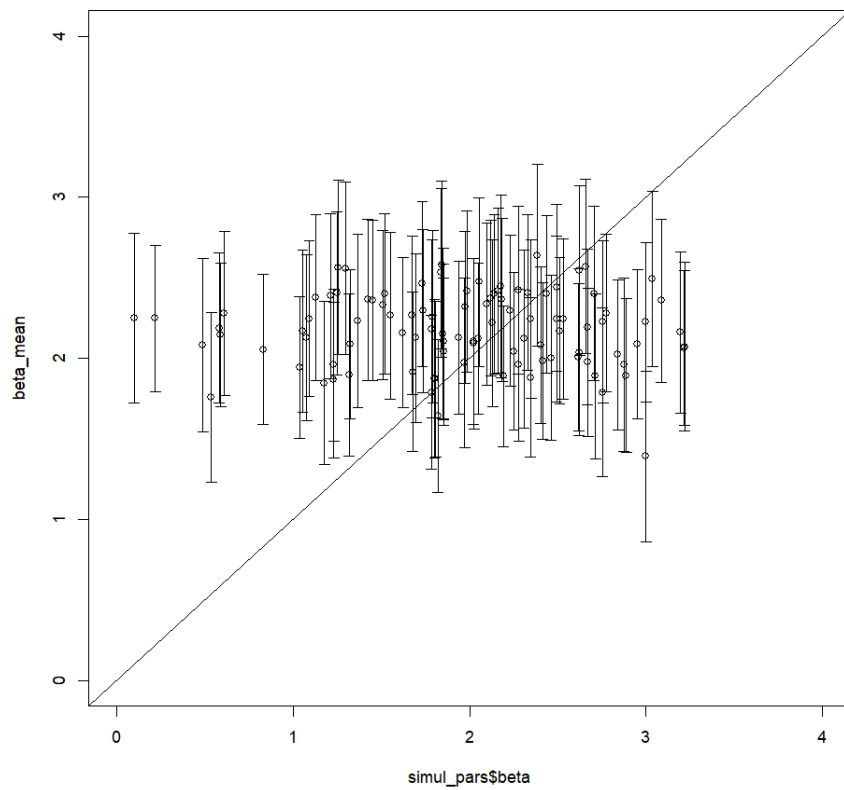
Also, shrinkage in alpha seems noticeably greater than that of beta. This is possibly due to the fact that information obtained from the individual-level data is a lot less for alpha compared to beta. While beta (inverse temperature) can be estimated from all trials across all subjects, additional information for alpha (learning rate) is only given when there is a prediction error across trials. Hence, posterior alpha is more heavily influenced by the group mean due to less abundant information from individual data, resulting in greater shrinkage towards the group mean, while posterior beta is less affected by the group mean.



Question 4

(2)

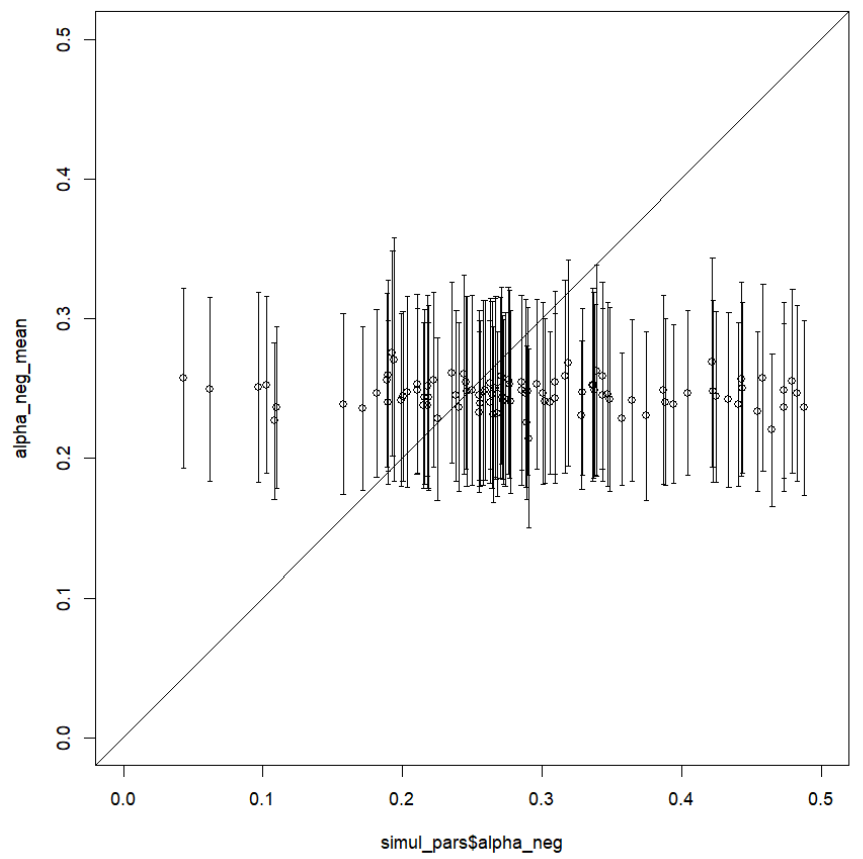
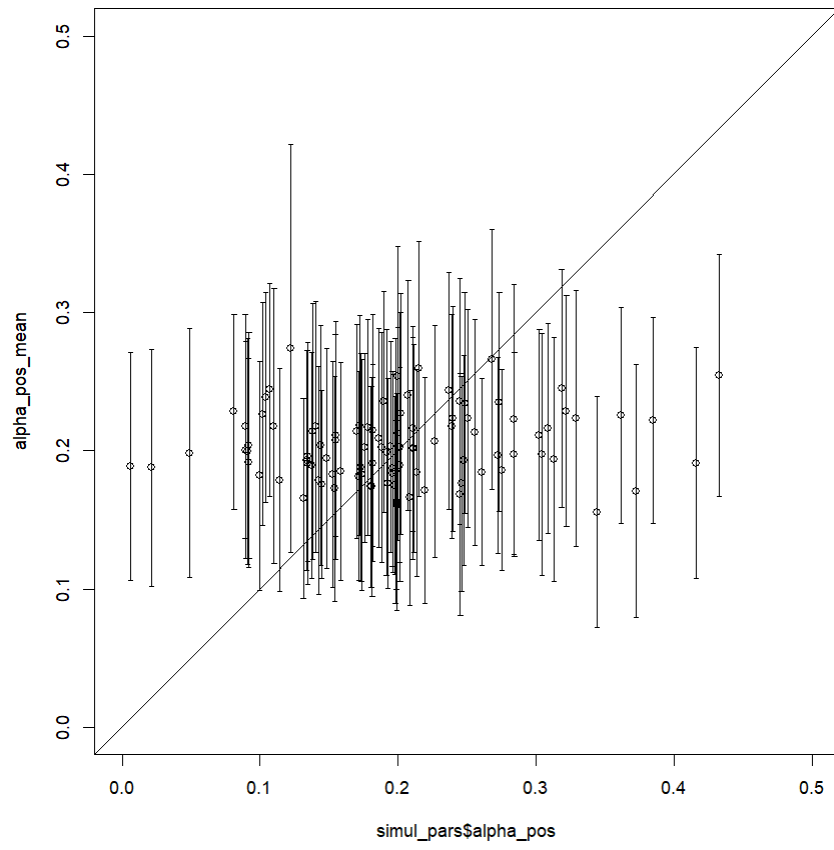


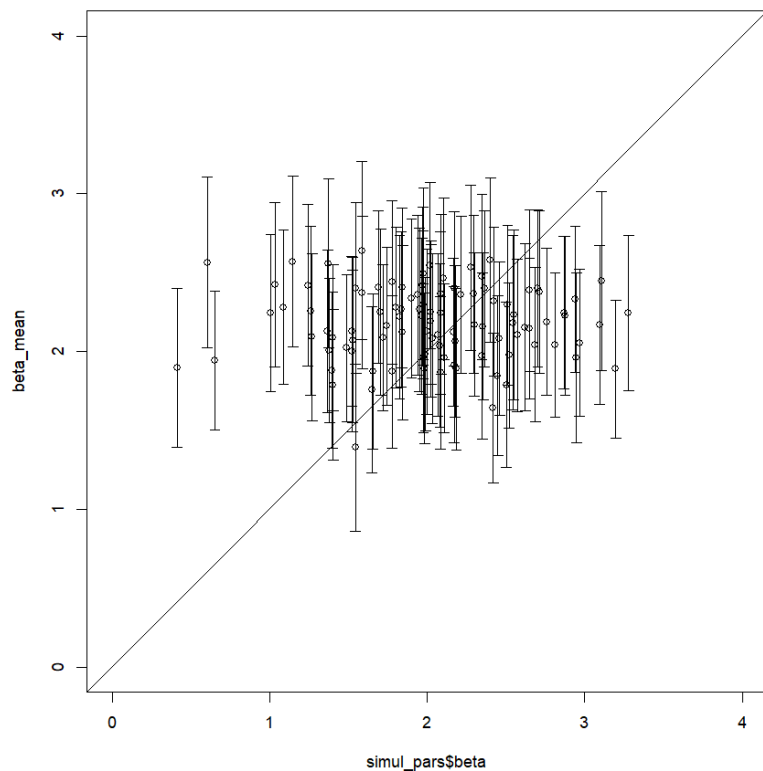


- Additionally, posterior summary for the group parameters

	mean	se_mean	sd	2.5%	50%	97.5%	n_eff	Rhat
mu_alpha_pos	0.19	0	0.04	0.13	0.19	0.26	643	1
	mean	se_mean	sd	2.5%	50%	97.5%	n_eff	Rhat
mu_alpha_neg	0.24	0	0.03	0.19	0.24	0.3	2550	1
	mean	se_mean	sd	2.5%	50%	97.5%	n_eff	Rhat
mu_beta	2.17	0.01	0.22	1.82	2.14	2.68	546	1

(3)





- Additionally, posterior summary for the group parameters

	mean	se_mean	sd	2.5%	50%	97.5%	n_eff	Rhat
mu_alpha_pos	0.17		0	0.02	0.14	0.17	0.2	267
								1
	mean	se_mean	sd	2.5%	50%	97.5%	n_eff	Rhat
mu_alpha_neg	0.31		0	0.01	0.29	0.31	0.34	1118
								1
	mean	se_mean	sd	2.5%	50%	97.5%	n_eff	Rhat
mu_beta	2.07	0.01	0.14	1.79	2.07	2.35	281	1.01

(4) Unfortunately, no drastic differences between scatterplots from (2) and (3) are visibly noticeable. Although the given conditions in (3) had lower reward probabilities (0.3 vs. 0.6), the difference between the expected values of the two options remained similar to the original condition (0.5 vs. 0.8). The increased number of trials in (3) (300 instead of 100) was expected to have reduced posterior uncertainty by providing more behavioral data for estimation but may not have been visually apparent due to a ceiling effect (parameters were recovered already pretty well with 100 trials). As a result, the overall parameter recovery patterns and posterior uncertainties appeared relatively similar across the two conditions.

Question 5

Posterior distributions:

Somehow for both (1), (2) the summary statistics indicated uniform posteriors. Error wasn't able to be handled in time.

Code for (1):

```
N = 5 # number of subjects
```

```
T = 140 # number of trials per subject
```

```
# dataset
```

```
data <- read.table("ra_exampleData.txt", header = TRUE)
```

```
# model (ra_prospect)
```

```
ra_prospect <- function(param, cert, gain, loss, gamble, T){  
  likelihood = 1
```

```
  for (t in 1:T) {
```

```
    # cert[t]: outcome of a certain option on trial t
```

```
    # gain[t] & loss[t]: gain and loss outcome of a risky option on trial t
```

```
    # gamble[t]: choice on trial t. gamble[t]=1 --> chose a risky option. 0 --> chose a certain option.
```

```
    #
```

```
    # evSafe: expected value of a certain (safe) option
```

```
    # evGamble: expected value of a risky option (gamble)
```

```
    # pGamble # probability of choosing a gamble on each trial
```

```
    # free parameters: rho, tau, lambda
```

```
    evSafe = cert[t]^param[1]
```

```
    evGamble = 0.5*(gain[t]^param[1] - param[2]*abs(loss[t])^param[1])
```

```
    pGamble = 1 / (1 + exp(param[3]*(evSafe - evGamble)))
```

```
    pGamble = pGamble * 0.9998 + 0.0001 # to make its range between 0.0001 and 0.9999
```

```
    tmp_like = (pGamble)^gamble[t] * (1-pGamble)^(1-gamble[t]) # likelihood of trial t  
(instead of minus log likelihood)
```

```
    likelihood = likelihood * tmp_like
```

```
  }
```

```
  likelihood
```

```
}
```

```
# define grid
```

```
rho_grid <- seq( from=0, to=2, length.out=10 )
```

```
lambda_grid <- seq( from=0, to=5, length.out=10 )
```

```
tau_grid <- seq( from=0, to=10, length.out=10 )
```

```

# prior (uniform prior)
prior <- array(1, dim=c(10,10,10))

posterior <- vector("list", N)

for (s in 1:N){

  curr_data <- data[data$subjID == s, ]

  likelihood <- array(NA, dim=c(10,10,10))

  for (i in 1:10){
    for (j in 1:10){
      for (k in 1:10){
        param <- c(
          rho_grid[i],
          lambda_grid[j],
          tau_grid[k]
        )
        # likelihood
        likelihood[i,j,k] <-
          ra_prospect(
            param,
            curr_data$cert,
            curr_data$gain,
            curr_data$loss,
            curr_data$gamble,
            T
          )
      }
    }
  }

  # product of likelihood and prior
  unstd.posterior <- likelihood * prior

  # standardize the posterior, so it sums to 1
  posterior[[s]] <- unstd.posterior / sum(unstd.posterior)
}

```